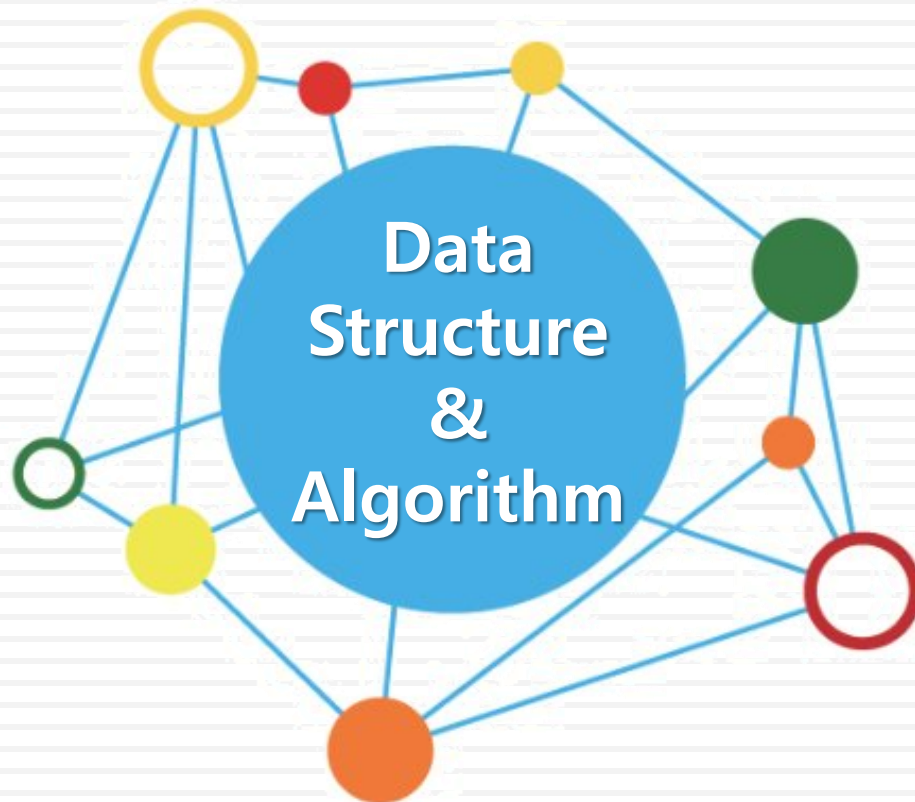


데이터 구조론

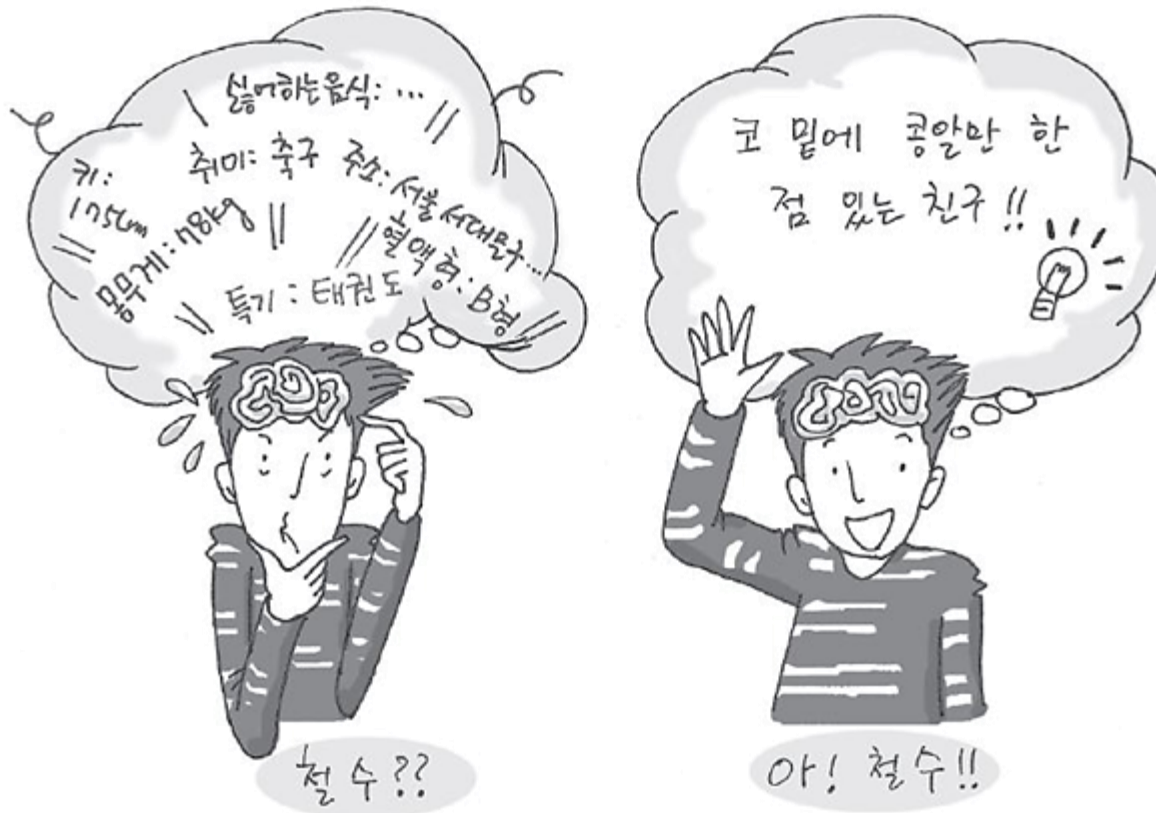


유길현

데이터 구조와 알고리즘

❖ 뇌의 추상화 기능

- 기억할 대상의 구별되는 특징만을 단순화하여 기억하는 기능



❖ 컴퓨터를 이용한 문제해결에서의 추상화

- 크고 복잡한 문제를 단순화시켜 쉽게 해결하기 위한 방법
- 자료 추상화(Data Abstraction)
 - 처리할 자료, 연산, 데이터에 대한 추상화 표현
 - 자료 : 프로그램의 처리 대상이 되는 모든 것을 의미
 - 연산
 - 어떤 일을 처리하는 과정. 연산자에 의해 수행
 - 연산자 : 연산자 집합. $\{+, -, \times, \div, \text{mod}\}$
 - 데이터형
 - 처리할 데이터의 집합과 자료에 대해 수행할 연산자의 집합
 - 예) 정수 자료형 : 정수의 집합. $\{\dots, -1, 0, 1, \dots\}$

❖ 추상 데이터형(ADT, Abstract Data Type)

- 데이터와 연산자의 특성을 논리적으로 추상화하여 정의한 데이터형
- 추상화와 구체화
 - 추상화 – “무엇(what)인가?”를 논리적으로 정의
 - 구체화 – “어떻게(how) 할 것인가?”를 실제적으로 표현
 - 데이터와 연산에 있어서 추상화와 구체화의 관계

구분	데이터	연산
추상화	추상 데이터형	알고리즘 정의
구체화	데이터형	프로그램 구현

❖ 알고리즘은 문제 해결 과정을 묘사하는 것

- 알고리즘은 문제 해결 절차를 체계적으로 기술한 것이다.
- 문제의 요구조건
 - 입력과 출력으로 명시 할 수 있다.
 - 알고리즘은 입력을 받아 출력을 만들어 내는 과정을 기술한다.



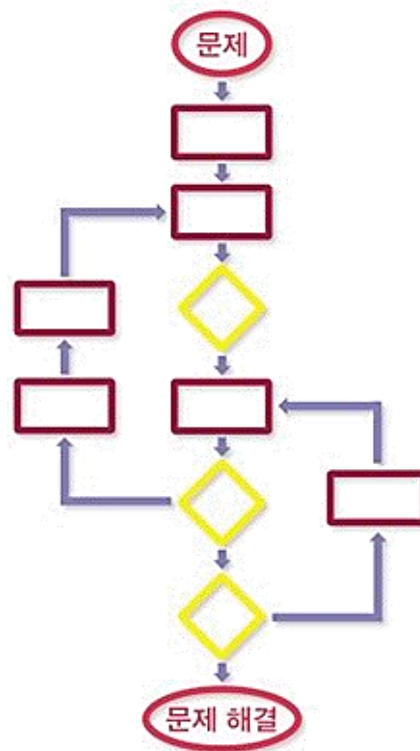
❖ 알고리즘은 생각하는 방법을 훈련 하는 것

- 문제를 해결 처리 방법, 절차등의 과정에 깃들여 있는 생각하는 방법을 배운다.
- 문제를 해결하는데 검증된 방법, 절차는 다른 문제를 해결할 때 사용하는 사고의 빌딩 블록을 제공한다.



❖ 알고리즘은 데이터구조의 확장이다.

- 데이터구조는 데이터를 어떻게 저장하고, 어떻게 탐색할 것인지 어떠한 방법으로 삭제 할 것인가를 결정 해야 한다.
- 데이터를 저장, 탐색, 삭제하는 방법과 절차등의 문제 해결을 하기 위해서는 알고리즘이 필요하다.



❖ 딸기 시럽을 얹은 치즈케이크 만들기

요리 재료 데이터



케이크 시트(20cm×20cm) 1개, 크림치즈 무스(크림치즈 200g, 달걀 2알, 설탕 3큰술, 레몬즙 1큰술, 바닐라 에센스 1큰술), 딸기 시럽(딸기 500g, 설탕 1½컵, 레몬즙 1작은술), 딸기 1개, 플레인 요구르트 2큰술

요리법

알고리즘

절차

- 1 케이크 틀에 유산지를 깔고 케이크 시트를 놓는다.
- 2 달걀 2알을 잘 푼다. 볼에 크림치즈를 넣고 거품기로 젓는다. 달걀 푼 물과 설탕 3큰술을 세 차례로 나누어 넣으면서 크림 상태가 되도록 거품기로 젓는다.
- 3 2에 레몬즙과 바닐라 에센스를 넣고 살짝 저은 다음 1에 붓는다. 180℃로 예열된 오븐에 전체를 넣고 20분 정도 굽는다.
- 4 딸기를 얇게 자르고 냄비에 넣은 다음 설탕 1½컵을 넣고 약한 불로 끓인다. 끓어붙지 않도록 계속해서 젓고 거품이 생기면 건어 낸다. 되직해지면 레몬즙을 넣고 차갑게 식힌다.
- 5 치즈케이크 한 조각을 접시에 담고 4를 뿌린 다음 플레인 요구르트와 딸기를 얹는다.

연산

❖ 알고리즘

- 문제해결 방법의 단계적 절차를 논리적으로 기술한 것
- 알고리즘의 조건

조건	내용
입력 (Input)	알고리즘 수행에 필요한 자료가 외부에서 입력으로 제공될 수 있어야 한다.
출력 (output)	알고리즘 수행 후 하나 이상의 결과를 출력해야 한다.
명확성 (definiteness)	수행할 작업의 내용과 순서를 나타내는 알고리즘의 명령어들은 명확하게 명시되어야 한다.
유한성 (finiteness)	알고리즘은 수행 뒤에 반드시 종료되어야 한다.
효율성 (effectiveness)	알고리즘의 모든 명령어들은 기본적인 실행이 가능해야 한다

❖ 알고리즘의 기술 방법

- 영어나 한국어와 같은 자연어로 작성
- 흐름도(flow chart)로 표현
- 유사 코드(pseudo-code)로 작성
- C와 같은 프로그래밍 언어로 작성

❖ 자연어를 이용한 서술적 표현방법

- 인간이 읽기가 쉽다.
- 자연어의 단어들을 정확하게 정의하지 않으면 의미 전달이 모호해질 우려가 있다.

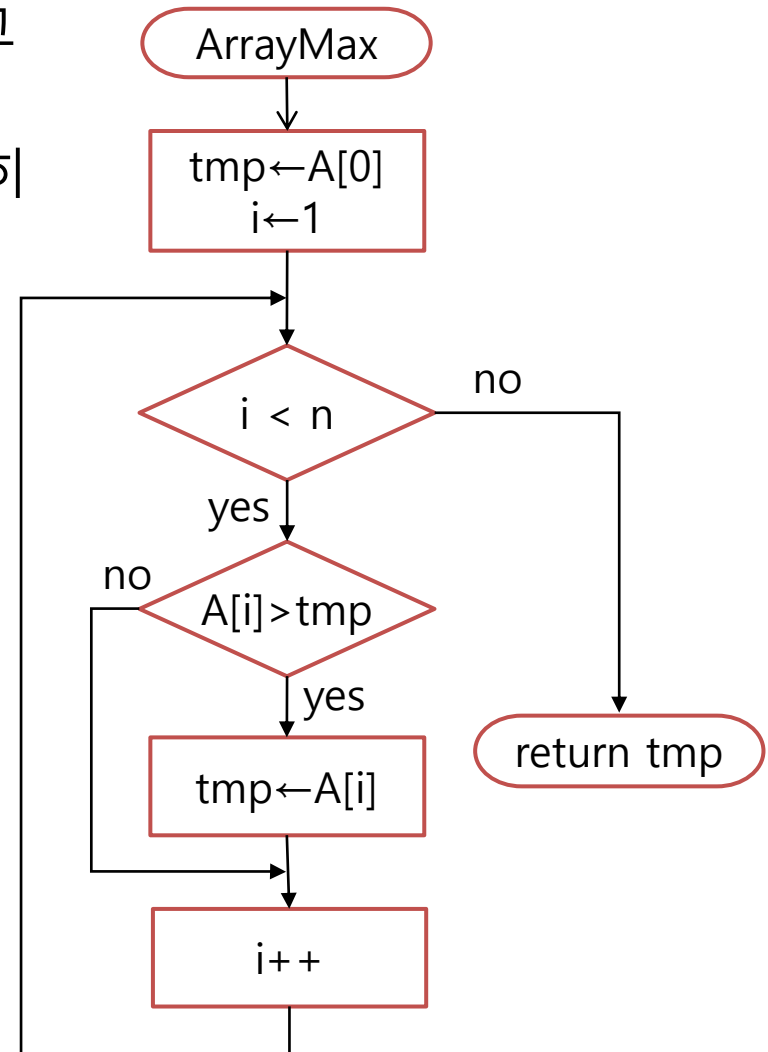
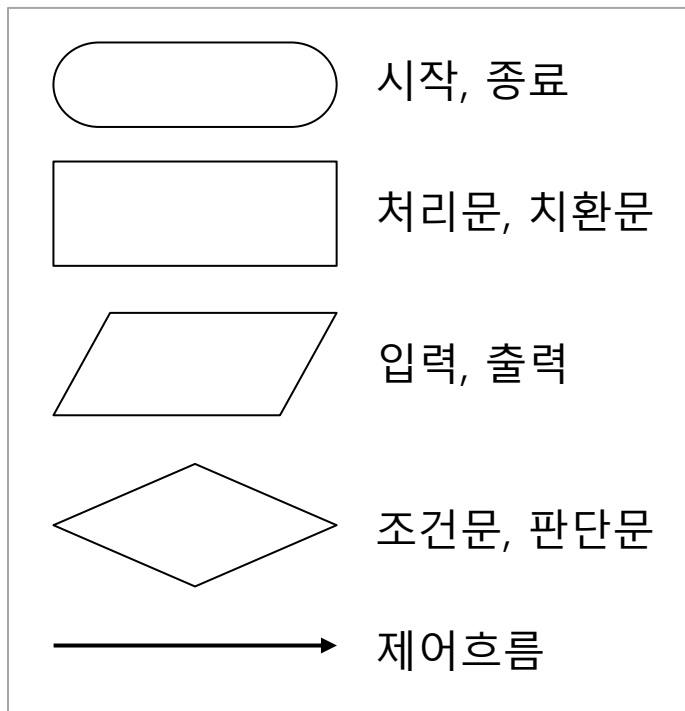
예) 배열에서 최대값 찾기 알고리즘

ArrayMax(A,n)

1. 배열 A의 첫 번째 요소를 변수 tmp에 복사
2. 배열 A의 다음 요소들을 차례대로 tmp와 비교하면 더 크면 tmp로 복사
3. 배열 A의 모든 요소를 비교했으면 tmp를 반환

❖ 순석도(Flow chart)를 이용한 도식화 표현 방법

- 직관적이고 이해하기 쉬운 알고리즘 기술 방법
- 복잡한 알고리즘의 경우, 상당히 복잡해짐.



❖ 유사코드를 이용한 표현 방법

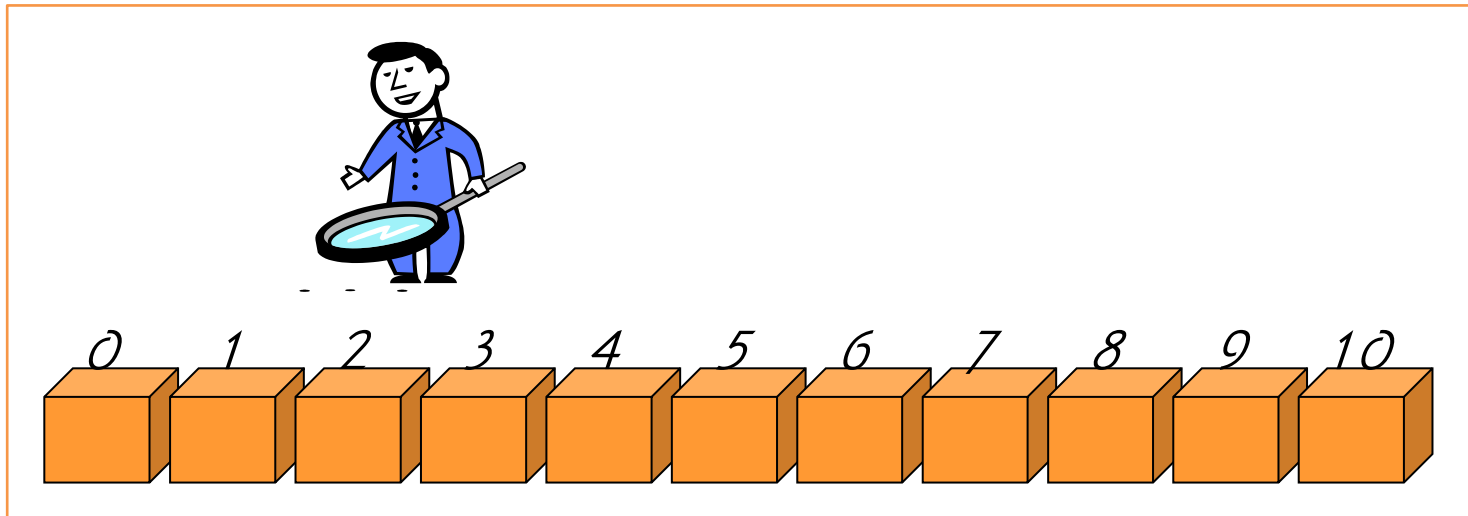
- 고수준의 알고리즘 기술 방법
- 자연어보다는 더 구조적인 표현 방법
- 프로그래밍 언어보다는 덜 구체적인 표현방법
- 알고리즘 기술에 가장 많이 사용
- 알고리즘의 핵심적인 내용에만 집중할 수 있다.

```
ArrayMax(A,n)
```

```
tmp ← A[0];  
for i←1 to n-1 do  
    if tmp < A[i] then  
        tmp ← A[i];  
return tmp;
```

❖ C 언어를 이용한 표현 방법

- 알고리즘의 가장 정확한 기술이 가능하다.
- 실제 구현 시, 많은 구체적인 사항들이 알고리즘의 핵심적인 내용에 대한 이해를 방해할 수 있다.
- 배열에서 최대값 찾기 알고리즘



❖ C 언어를 이용한 표현 방법

- 배열에서 최대값 찾기 알고리즘

```
#define ELEMENTS 10
int score[ELEMENTS];
int find_max_score(int n)
{
    int i, tmp;
    tmp=score[0];
    for(i=1;i<n;i++){
        if( score[i] > tmp ){
            tmp = score[i];
        }
    }
    return tmp;
}
```

❖ 배열에서 최대값 찾기

- 10개의 배열에 자동으로 난수를 입력하고 배열에 입력된 수 중 가장 큰 값을 찾아 화면에 출력하기
- 난수 발생은 rand() 함수를 사용하고, 프로그램 실행 시 동일 패턴의 난수 발생을 피하기 위해 srand((unsigned int)time(NULL)) 함수를 사용한다.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define ELEMENTS 10
int score[ELEMENTS];
int find_max_score(int n);
```


❖ 배열에서 최대값 찾기

- 난수를 발생시켜 배열에 저장

```
int main()
{
    srand((unsigned int)time(NULL));
    score[ELEMENTS] = { 0 };
    int i, maxscore;
    for (i = 0; i < ELEMENTS; i++)
    {
        score[i] = rand() % 99 + 1;
        printf("%2d ", score[i]);
    }
    maxscore = find_max_score(ELEMENTS);
    printf("\n\n최대값 = %d", maxscore);
    getchar();
    return 0;
}
```

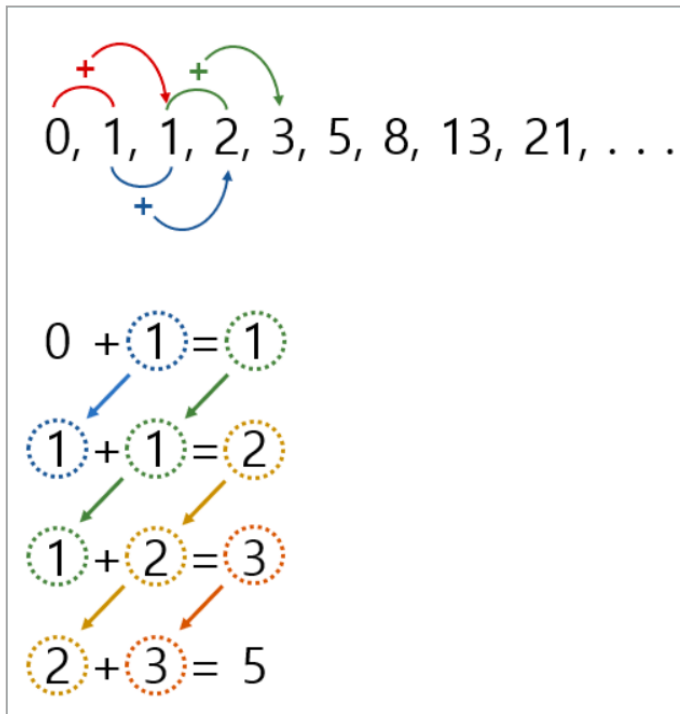
❖ 배열에서 최대값 찾기

- 배열에 저장된 값 중 가장 큰 값을 찾는다.

```
int find_max_score(int n)
{
    int i, tmp;
    tmp = score[0];
    for (i = 1; i < n; i++) {
        if (score[i] > tmp) {
            tmp = score[i];
        }
    }
    return tmp;
}
```

❖ 피보나치 수열

- 수열이란 어떤 일정한 규칙에 따라 배열되어 있는 수들을 말함
- 피보나치 수열은 연속하여 나타낸 두수의 합이 다음 수가 되는 규칙을 가진 수열



$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

❖ 피보나치 수열

- 수를 입력 받아 피보나치 수열 값으로 나타내기

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"

int main()
{
    int num;
    int value1 = 0, value2 = 1;
    int printValue = 0;
    int cnt = 0;

    printf("몇 항까지 출력하는가(0 이상의 수) : ");
    scanf("%d", &num);
```

❖ 피보나치 수열

- 수를 입력 받아 피보나치 수열 값으로 나타내기

```
while (cnt != num)
{
    printf("%d ", printValue); // 출력될 값
    printValue = value2;       // 두 번째 value 값을 출력하기 위해
    value2 += value1;          // 두 번째 값에 첫 번째 값을 더한다
    value1 = printValue;       // 첫 번째 값에 두 번째 값을 대입한다.
    cnt++;
}
printf("\n");

return 0;
}
```

❖ 수를 입력 받아 피보나치 수열 값으로 나타내기

- 재귀호출 함수를 이용

```
#include "stdio.h"

int getFibo(int n);

int main()
{
    int num, i;
    printf("몇 항까지 출력하는가(0이상의 수): ");
    scanf("%d", &num);

    for (i = 0; i <= num; i++)
        printf("%d ", getFibo(i));

    printf("\n");
    return 0;
}
```

❖ 수를 입력 받아 피보나치 수열 값으로 나타내기

- 재귀호출 함수를 이용

```
int getFibo(int n)
{
    if (n < 2)
        return n;
    else
        return getFibo(n - 1) + getFibo(n - 2);
}
```

❖ 알고리즘 성능 분석 기준

분석 기준	내 용
정확성	올바른 자료 입력 시 유한한 시간 내에 올바른 결과 출력 여부
단순성	알고리즘이 단순하면 정확성을 증명하기 쉬움
작업량	일반적인 연산 제외, 알고리즘 특성 나타내는 중요 연산 모두 분석
기억장소 사용량	메모리 사용량
최적성	최적이라는 것은 '가장 좋은' 의미임

❖ 알고리즘 복잡도 분석

복잡도	내 용
공간 복잡도	- 알고리즘을 프로그램으로 실행하여 완료하기까지 필요한 총 저장 공간의 양 - 공간 복잡도 = 고정 공간 + 가변 공간
시간 복잡도	- 알고리즘을 프로그램으로 실행하여 완료하기까지의 총 소요시간 - 시간 복잡도 = 컴파일 시간 + 실행 시간 ☞ 컴파일 시간 : 프로그램마다 거의 고정적인 시간 소요 ☞ 실행 시간 : 컴퓨터의 성능에 따라 달라질 수 있으므로 실제 실행시간 보다는 명령문의 실행 빈도수에 따라 계산 - 실행 빈도수의 계산 ☞ 지정문, 조건문, 반복문 내의 제어문과 반환문은 실행 시간 차이가 거의 없으므로 하나의 단위시간을 갖는 기본 명령문으로 취급

❖ 시간 복잡도 분석의 예 : n 을 n 번 더하는 문제

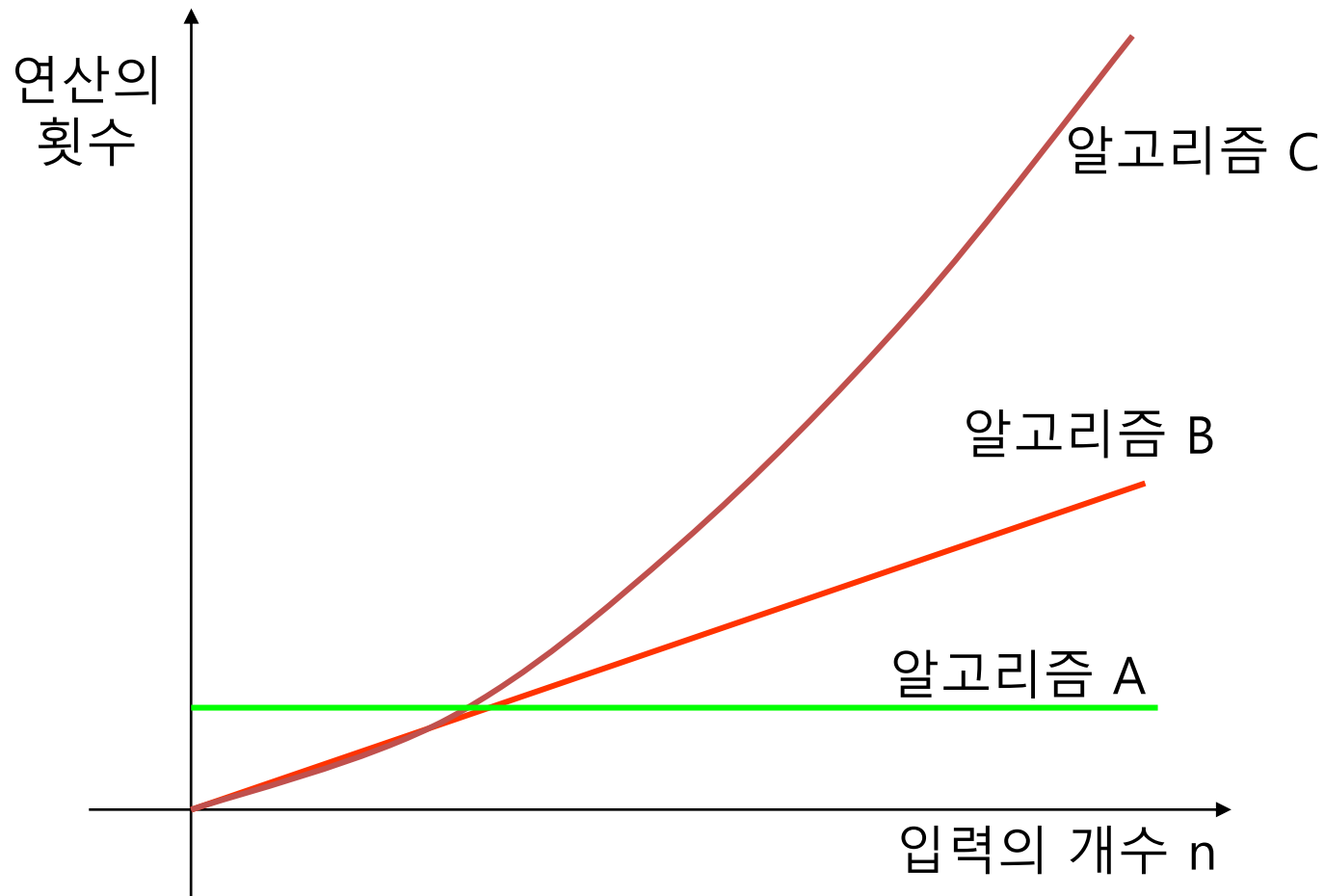
- 각 알고리즘이 수행하는 연산의 개수를 세어 본다.
- 단, for 루프 제어 연산은 고려하지 않음.

알고리즘 A	알고리즘 B	알고리즘 C
$\text{sum} \leftarrow n * n;$	$\text{sum} \leftarrow 0;$ $\text{for } i \leftarrow 1 \text{ to } n \text{ do}$ $\quad \text{sum} \leftarrow \text{sum} + n;$	$\text{sum} \leftarrow 0;$ $\text{for } i \leftarrow 1 \text{ to } n \text{ do}$ $\quad \text{for } j \leftarrow 1 \text{ to } n \text{ do}$ $\quad \quad \text{sum} \leftarrow \text{sum} + 1;$

	알고리즘 A	알고리즘 B	알고리즘 C
대입연산	1	$n + 1$	$n * n + 1$
덧셈연산		n	$n * n$
곱셈연산	1		
나눗셈연산			
전체연산수	2	$2n + 1$	$2n^2 + 1$

❖ 시간 복잡도 분석의 예 : n 을 n 번 더하는 문제

- 연산의 횟수를 그래프로 표현



❖ 시간 복잡도 계산의 예

- 코드를 분석해보면 수행되는 연산들의 횟수를 입력 크기의 함수로 만들 수 있다.

ArrayMax(A,n)

```
tmp ← A[0];  
for i ← 1 to n-1 do  
    if tmp < A[i] then  
        tmp ← A[i];  
return tmp;
```

1번의 대입 연산
루프 제어 연산은 제외
n-1번의 비교 연산
n-1번의 대입 연산(최대)
1번의 반환 연산

총 연산수 = $2n$ (최대)

❖ 빅-오 표기법

- 자료의 개수가 많은 경우에는 차수가 가장 큰 항이 가장 영향을 크게 미치고 다른 항들은 상대적으로 무시될 수 있다.
- 예) $n=1,000$ 일 때, $T(n)$ 의 값은 1,001,001이고 이중에서 첫 번째 항인 n^2 의 값이 전체의 약 99%인 1,000,000이고 두 번째 항의 값이 1000으로 전체의 약 1%를 차지한다.
- 따라서 보통 시간복잡도 함수에서 가장 영향을 크게 미치는 항만을 고려하면 충분하다.

$n=1000$ 인 경우

$$T(n) = n^2 + n + 1$$

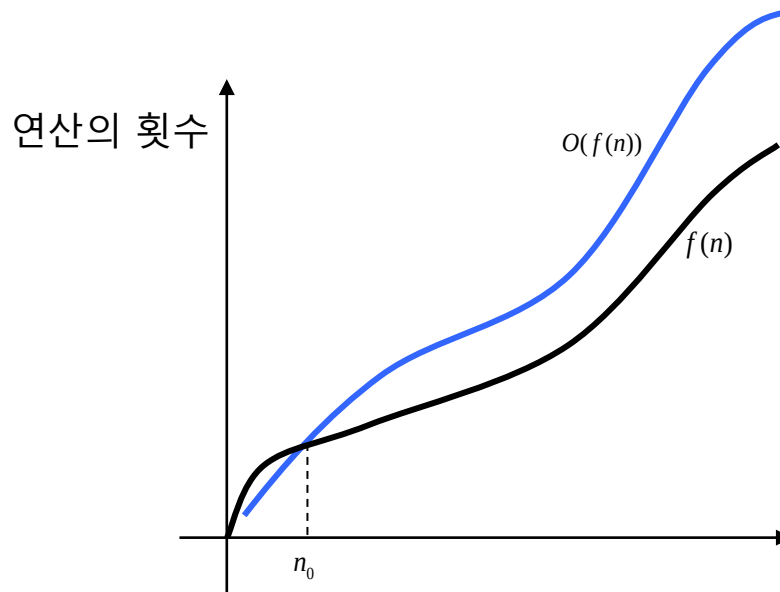
99%

1%

❖ 빅-오 표기법

- 연산의 횟수를 대략적(점근적)으로 표기한 것
- 두 개의 함수 $f(n)$ 과 $g(n)$ 이 주어졌을 때, 모든 $n \geq n_0$ 에 대하여 $|f(n)| \leq c |g(n)|$ 을 만족하는 2개의 상수 c 와 n_0 가 존재하면 $f(n) = O(g(n))$ 이다.
- 빅-오는 함수의 **상한을** 표시한다.

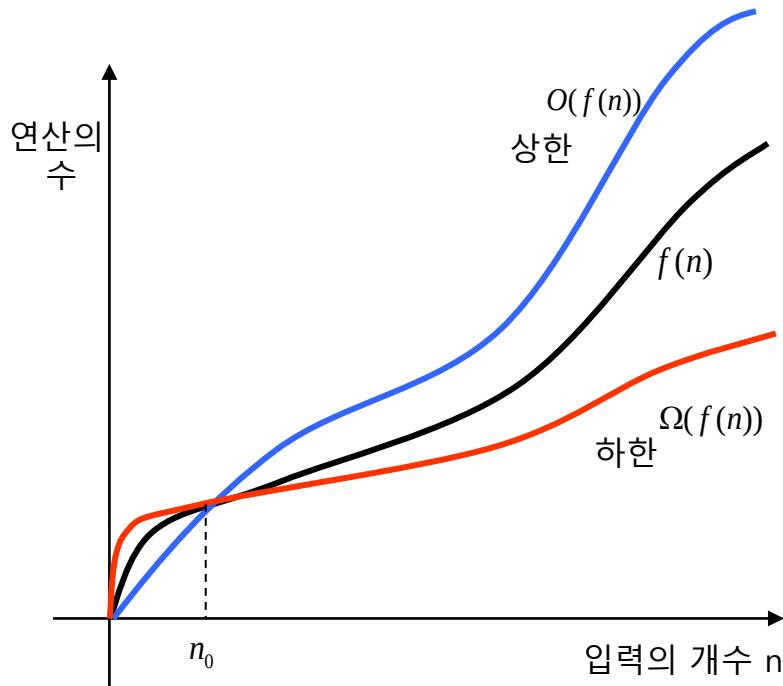
예) $n \geq 5$ 이면 $2n+1 < 10n$ 이므로 $2n+1 = O(n)$



❖ 빅-오메가 표기법

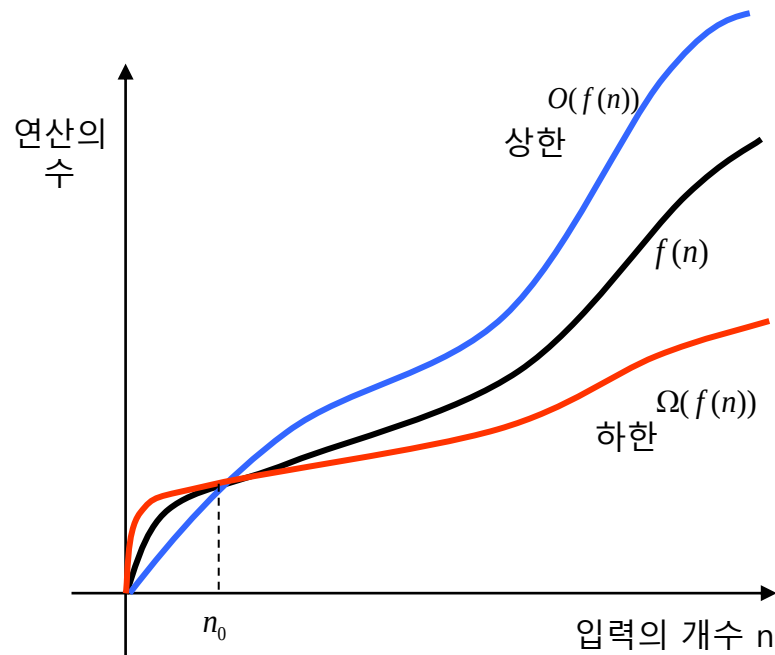
- 모든 $n \geq n_0$ 에 대하여 $|f(n)| \geq c |g(n)|$ 을 만족하는 2개의 상수 c 와 n_0 가 존재하면 $f(n) = \Omega(g(n))$ 이다.
- 빅오메가는 함수의 **하한을** 표시한다.

(예) $n \geq 5$ 이면 $2n+1 < 10n$ 이므로 $n = \Omega(n)$



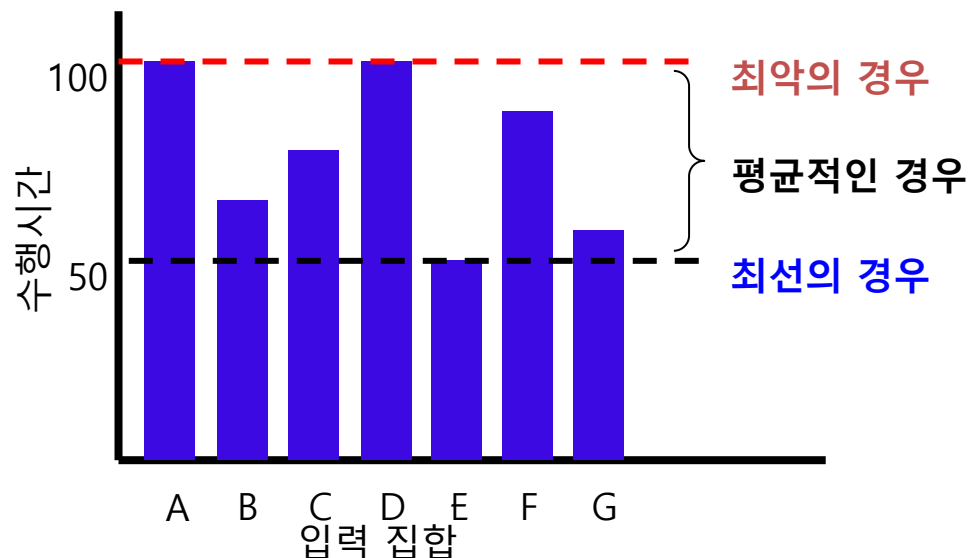
❖ 빅-세타 표기법

- 모든 $n \geq n_0$ 에 대하여 $c_1|g(n)| \leq |f(n)| \leq c_2|g(n)|$ 을 만족하는 3개의 상수 c_1 , c_2 와 n_0 가 존재하면 $f(n) = \theta(g(n))$ 이다.
- 빅세타는 함수의 **하한인 동시에 상한을 표시한다**.
- $f(n) = O(g(n))$ 이면서 $f(n) = \Omega(g(n))$ 이면 $f(n) = \theta(n)$ 이다.
(예) $n \geq 1$ 이면 $n \leq 2n+1 \leq 3n$ 이므로 $2n+1 = \theta(n)$



❖ 알고리즘의 수행시간

- 수행시간은 입력 자료 집합에 따라 다를 수 있다.
예) 정렬 알고리즘의 수행 시간은 입력 집합에 따라 다를 수 있다.
- 최선의 경우(best case): 수행 시간이 가장 빠른
- 평균의 경우(average case): 수행시간이 평균적인 경우
- 최악의 경우(worst case): 수행 시간이 가장 늦은 경우



❖ 순차탐색의 예

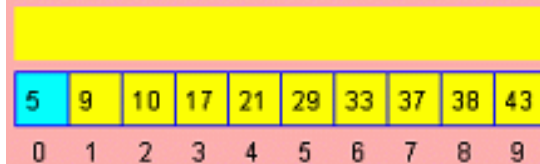
■ 최선의 경우(best case)

찾고자 하는 숫자가 맨 앞에 있는 경우

$$\therefore O(1)$$

※ 의미가 없는 경우가 많음

인덱스 0에서 값 5 발견
숫자 비교 횟수 = 1



■ 최악의 경우(worst case)

찾고자 하는 숫자가 맨 뒤에 있는 경우

$$\therefore O(n)$$

※ 가장 널리 사용

인덱스 9에서 값 43 발견
숫자 비교 횟수 = 10



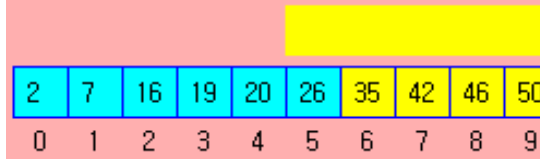
■ 평균적인 경우(average case)

각 요소들이 균일하게 탐색된다고
가정하면 $(1+2+\dots+n)/n = (n+1)/2$

$$\therefore O(n)$$

※ 계산하러 복잡하고 가정을 제대로 정의
하지 못하면 신뢰도가 낮아짐

인덱스 5에서 값 26 발견
숫자 비교 횟수 = 6



❖ 순차탐색의 시간 복잡도 계산

```
# include<stdio.h>

int LinearSearch(int ar[], int len, int target)
{
    int i;
    for (i = 0; i < len; i++)
    {
        if (ar[i] == target)
            return i;
    }
    return -1;
}
```

❖ 순차탐색의 시간 복잡도 계산

```
int main()
{
    int arr[] = { 3, 5, 2, 4, 9 };
    int idx;

    idx = LinearSearch(arr, sizeof(arr) / sizeof(int), 4);
    if (idx == -1)
        printf("탐색실패 \n");
    else
        printf("타겟 저장 인덱스: %d \n", idx);
    idx = LinearSearch(arr, sizeof(arr) / sizeof(int), 7);
    if (idx == -1)
        printf("탐색실패 \n");
    else
        printf("타겟 저장 인덱스: %d \n", idx);
    getchar();
    return 0;
}
```

❖ 순차탐색의 시간 복잡도 계산

- 순차 탐색 알고리즘인 LinearSearch 함수내의 for문에서 데이터 수 n 에 대한 시간 복잡도 $T(n)$ 을 계산
- 알고리즘에 사용된 연산자는 $<$, $++$, $==$ 세 가지임 이 중 값을 비교하는 $==$ 연산을 가장 적게 수행하는 탐색 알고리즘이 좋은 탐색 알고리즘
- 탐색 알고리즘의 핵심은 동등비교를 하는 비교연산에 있음
 - 비교연산을 적게 할수록 $<$ 연산과 $++$ 연산의 수행횟수가 줄게 됨(다른 연산들이 $==$ 연산에 의존적) 즉, $==$ 연산의 횟수를 대상으로 시간 복잡도를 분석하면 됨
- 찾고자 하는 배열의 값이 맨 앞에 있으면 비교연산의 수행횟수는 1이 됨 -> best case
- 찾고자 하는 배열의 값이 맨 뒤에 있으면 비교연산의 수행횟수는 n 이 됨 -> worst case (중요)
- Average case의 경우 합리적이거나 계산이 복잡하고 평균의 정의가 애매 -> 따라서 worst case로 계산

❖ 순차탐색의 시간 복잡도 계산

- 최악의 경우(worst case)
 - 데이터의 수가 n 일 때 최악의 경우는 n 이 됨
 - $T(n)=n$
- 평균의 경우(average case)
 - 가정 1. 탐색 대상이 배열에 존재하지 않을 확률을 50%라고 가정: $T(n)=n$
 - 가정 2. 배열의 첫 요소부터 마지막 요소까지 탐색 대상이 존재할 확률은 동일: $T(n)=n/2$
 - 가정 1과 가정 2에 따라 위의 두 식을 하나로 묶어야 함 -
 $> n \times 1/2 + n/2 \times 1/2 = 3/4n$
 - 따라서 순차 탐색 알고리즘의 average case의 시간 복잡도는 $T(n)=3/4n$ 가 됨
 - 계산이 복잡하고, 모든 가정을 제대로 정의하지 못하면 신뢰도가 낮게 됨

- 컴퓨터에서 수행시간을 측정하는 방법에는 주로 clock 함수가 사용
- clock_t clock(void);
 - clock 함수는 호출되었을 때의 시스템 시각을 CLOCKS_PER_SEC 단위로 반환
- 수행시간을 측정하는 전형적인 프로그램

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void main( void )
{
    clock_t start, finish;
    double  duration;
    start = clock();
    // 수행시간을 측정하고 하는 코드....
    // ....
    finish = clock();
    duration = (double)(finish - start) / CLOCKS_PER_SEC;
    printf("%f 초입니다.\n", duration);
}
```

❖ 알고리즘의 수행시간 측정

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
int algorithmA(int n)
{
    int sum;
    sum = n*(n + 1) / 2;
    return sum;
}
```


❖ 알고리즘의 수행시간 측정

```
int algorithmB(int n)
{
    int i, sum = 0;
    for (i = 1; i <= n; i++)
        sum = sum + i;
    return sum;
}
```

```
int algorithmC(int n)
{
    int i, k, sum = 0;
    for (i = 1; i <= n; i++)
        for (k = 0; k < i; k++)
            sum = sum + 1;
    return sum;
}
```

❖ 알고리즘의 수행시간 측정

```
int main()
{
    clock_t start, finish;
    long duration;
    int i, sum;

    start = clock();
    for (i = 0; i < 1500; i++)
        sum = algorithmA(1500);
    finish = clock();
    duration = (finish - start);
    printf("알고리즘 A: 합계: %d 소요시간: %ld\n", sum, duration);
}
```

❖ 알고리즘의 수행시간 측정

```
start = clock();
for (i = 0; i < 1500; i++)
    sum = algorithmB(1500);
finish = clock();
duration = (finish - start);
printf("알고리즘 B: 합계: %d 소요시간: %d\n", sum, duration);

start = clock();
for (i = 0; i < 1500; i++)
    sum = algorithmC(1500);
finish = clock();
duration = (finish - start);
printf("알고리즘 C: 합계: %d 소요시간: %d\n", sum, duration);

return 0;
}
```

❖ 비밀지도

길동이는 평소 철수가 비상금을 숨겨놓는 장소를 알려줄 비밀지도를 손에 넣었다. 그런데 이 비밀지도는 숫자로 암호화되어 있어 위치를 확인하기 위해서는 암호를 해독해야 한다. 다행히 지도 암호를 해독할 방법을 적어 놓은 메모도 함께 발견했다.

1. 지도는 한 변의 길이가 n 인 정사각형 배열 형태로, 각 칸은 “공백”(“ ”) 또는 “벽”(“#”) 두 종류로 이루어져 있다.
2. 전체 지도는 두 장의 지도를 겹쳐서 얻을 수 있다. 각각 “지도 1”과 “지도 2”라고 하자. 지도 1 또는 지도 2 중 어느 하나라도 벽인 부분은 전체 지도에서도 벽이다. 지도 1과 지도 2에서 모두 공백인 부분은 전체 지도에서도 공백이다.
3. “지도 1”과 “지도 2”는 각각 정수 배열로 암호화되어 있다.
4. 암호화된 배열은 지도의 각 가로줄에서 벽 부분을 1, 공백 부분을 0으로 부호화했을 때 얻어지는 이진수에 해당하는 값의 배열이다.

❖ 비밀지도

	#			#	$01001_{(2)} = 9$
#		#			$10100_{(2)} = 20$
#	#	#			$11100_{(2)} = 28$
#			#		$10010_{(2)} = 18$
	#		#	#	$01011_{(2)} = 11$
#	#	#	#		$11110_{(2)} = 30$
				#	$00001_{(2)} = 1$
#		#		#	$10101_{(2)} = 21$
#				#	$10001_{(2)} = 17$
#	#	#			$11100_{(2)} = 28$



#	#	#	#	#
#		#		#
#	#	#		#
#			#	#
#	#	#	#	#

❖ 비밀지도 프로그램 작성

■ 입력형식

- $1 \leq n \leq 16$
- arr1, arr2는 길이 n인 정수 배열로 주어진다.
- 정수 배열의 각 원소 x를 이진수로 변환했을 때의 길이는 n 이하이다. 즉, $0 \leq x < 2^n - 1$ 을 만족한다.

■ 출력 형식

원래의 비밀지도를 해독하여 “#”, 공백으로 구성된 문자열 배열로 출력

매개변수	값
n	5
arr1	[9, 20, 28, 18, 11]
arr2	[30, 1, 21, 7, 28]
출력	["#####", "# # #", "#### #", "# ##", "#####"]

❖ 비밀지도 프로그램 작성

■ 출력 형식

원래의 비밀지도를 해독하여 “#”, 공백으로 구성된 문자열 배열로 출력

매개변수	값
n	6
arr1	[46, 33, 33, 22, 31, 50]
arr2	[27, 56, 19, 14, 14, 10]
출력	["#####", "#### #", "## ##", " ##### ", " #####", "#### # "]

❖ 비밀지도 프로그램 코드

```
#include <stdio.h>
#define size 5

int main()
{
    int map1[size][size] = { 0 };
    int map2[size][size] = { 0 };
    int arr1[size] = { 9, 20, 28 ,18, 11};
    int arr2[size] = { 30, 1, 21, 17, 28};

    int i, j;
```


❖ 비밀지도 프로그램 코드

프로그램 코드 작성